

HTTP: 웹 통신의 기본 프로토콜

인터넷 통신의 핵심 기술인 HTTP에 대한 기술적 이해와 웹 개발에서의 활용에 대해 알아보겠습니다.



HTTP

(Hyper Text Transfer Protocol)

기본 개념

하이퍼텍스트 또는 하이퍼미디어 문서를 전송하는 프로토콜

웹 서버와 클라이언트(브라우저) 간 통신을 위한 클라이언트-서버 모델

특징

무상태(Stateless) 프로토콜

1990년대 초 설계

버전 발전

- HTTP/1.0, HTTP/1.1: 모든 웹사이트 지원
- HTTP/2: HTTPS에서 사용, 모든 사이트 지원
- HTTP/3: UDP 기반, 구글 기반 사이트 지원 중, 점차 확대



HTTP의 기본 개념

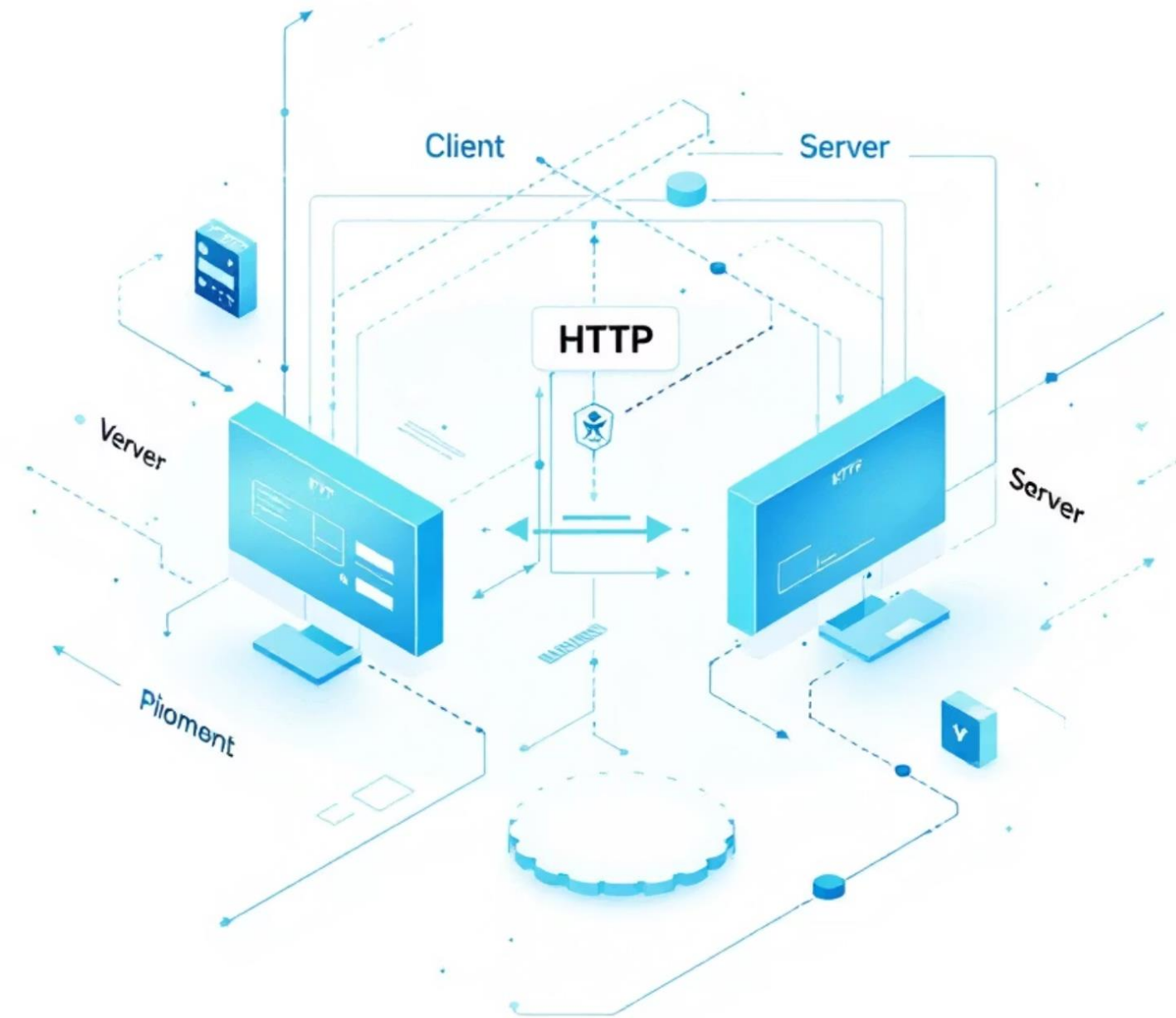
웹 통신의 기초

- HTTP는 HTML 문서와 같은 리소스들을 가져올 수 있도록 해주는 프로토콜입니다.
- 웹에서 이루어지는 모든 데이터 교환의 기본 토대가 됩니다.

클라이언트-서버 구조

- 수신자(서버) 측에 의해 요청이 초기화되는 프로토콜
- 웹 브라우저가 클라이언트 역할 수행
- 웹 서버가 요청에 응답하는 구조

클라이언트-서버 모델에서 HTTP는 요청과 응답의 형태로 동작합니다.

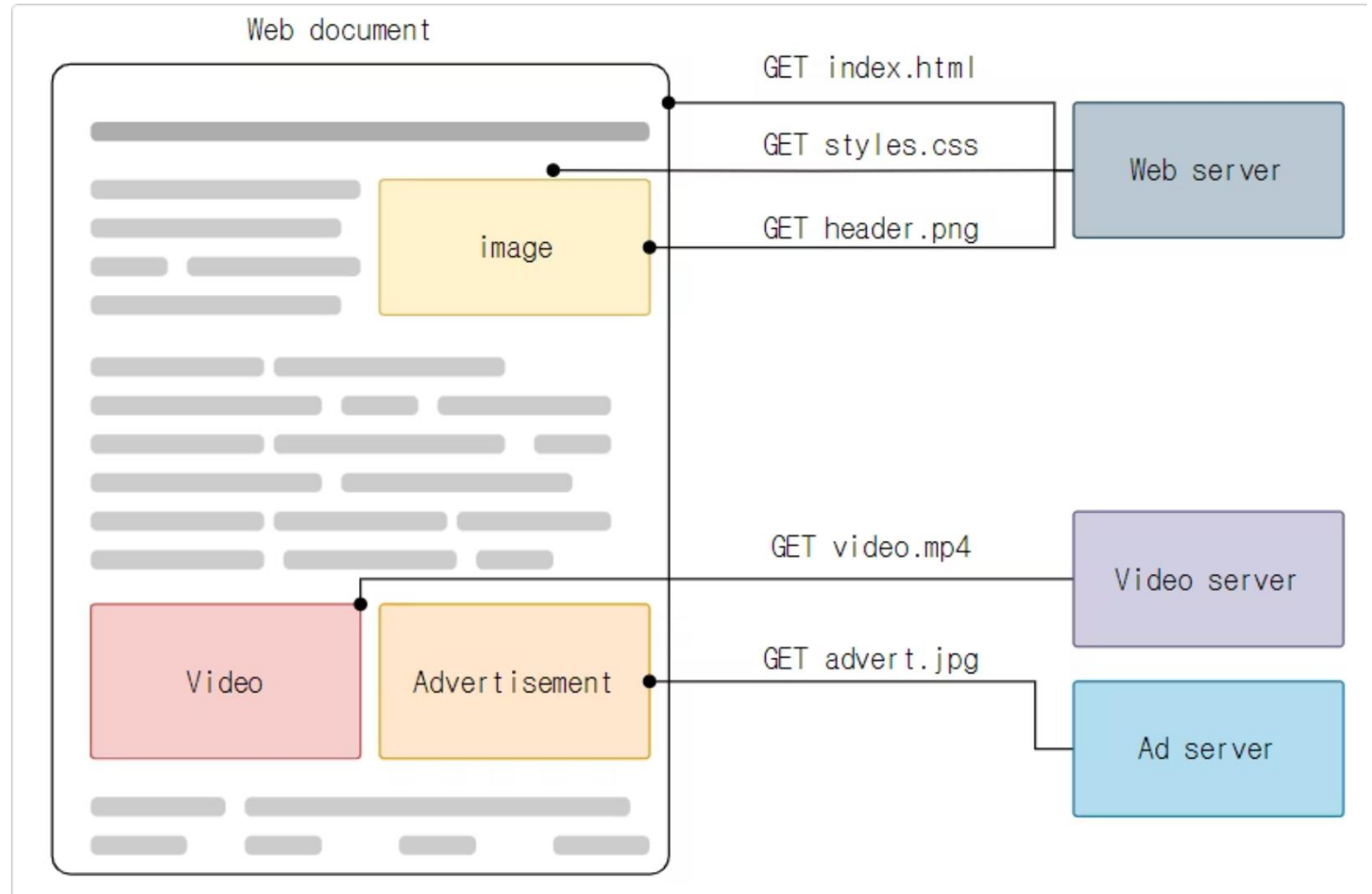


HTTP와 웹 문서 구성

하나의 완전한 문서 구성요소

- 텍스트 콘텐츠
- 레이아웃 설명 (CSS)
- 이미지 리소스
- 비디오 및 미디어 파일
- 스크립트 (JavaScript)

브라우저는 이러한 하위 문서들을 조합하여 사용자에게 완전한 웹페이지를 제공합니다.

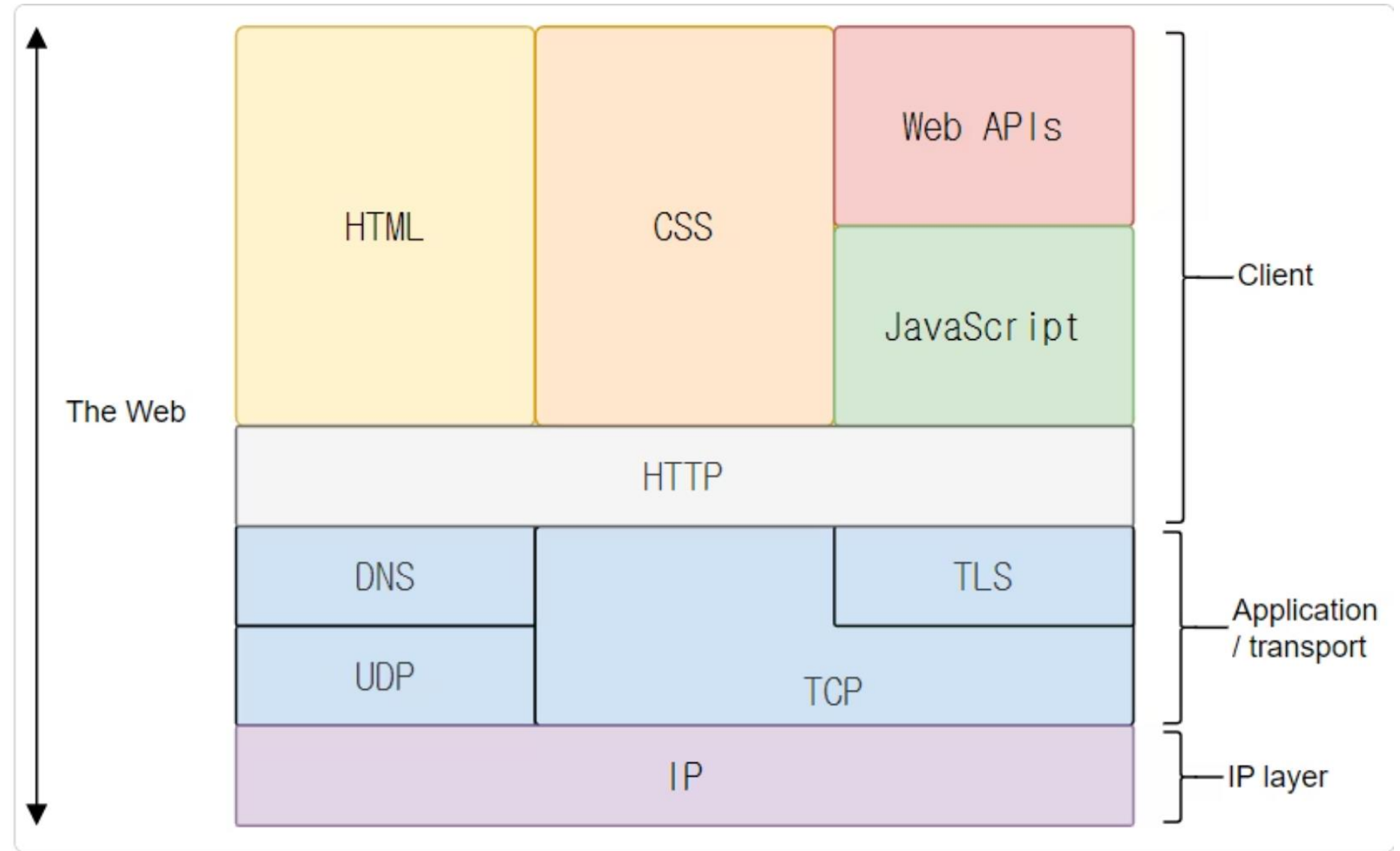


HTTP 메시지 교환

클라이언트-서버 간 통신

HTTP는 서버와 클라이언트 간의
메시지 교환을 통해 이루어집니다:

- 요청(Requests): 클라이언트가
서버로 전송하는 메시지
- 응답(Responses): 서버가
클라이언트로 전송하는 메시지



이 기본적인 요청-응답 구조는 모든 웹 통신의 기초가 됩니다.

HTTP 통신 구조



클라이언트 요청

Explorer나 Navigator와 같은 브라우저를 실행하는 PC나 Mac
HTTP 요청을 생성하여 서버로 전송



웹 서버 응답

Apache 웹 서버를 실행하는 서버 시스템
HTTP 요청을 처리하고 HTTP 응답 생성



응답 처리

클라이언트가 HTTP 응답을 받아 처리
브라우저가 HTML을 렌더링하여 사용자에게 표시

Request, connect in web s

HTTP request response

ing request is
omputment,
esponse

oid rerefct in
[.net](#)

HT
U\parater th
puspients
[new/](#)

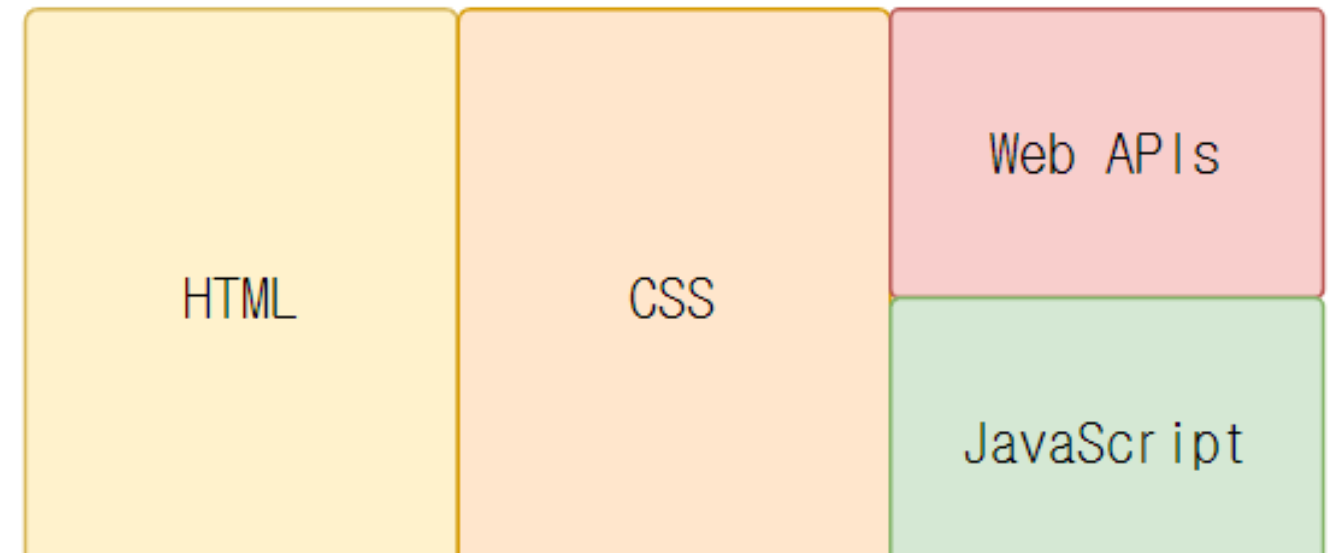
HTTP 기반 시스템: 클라이언트

사용자 에이전트

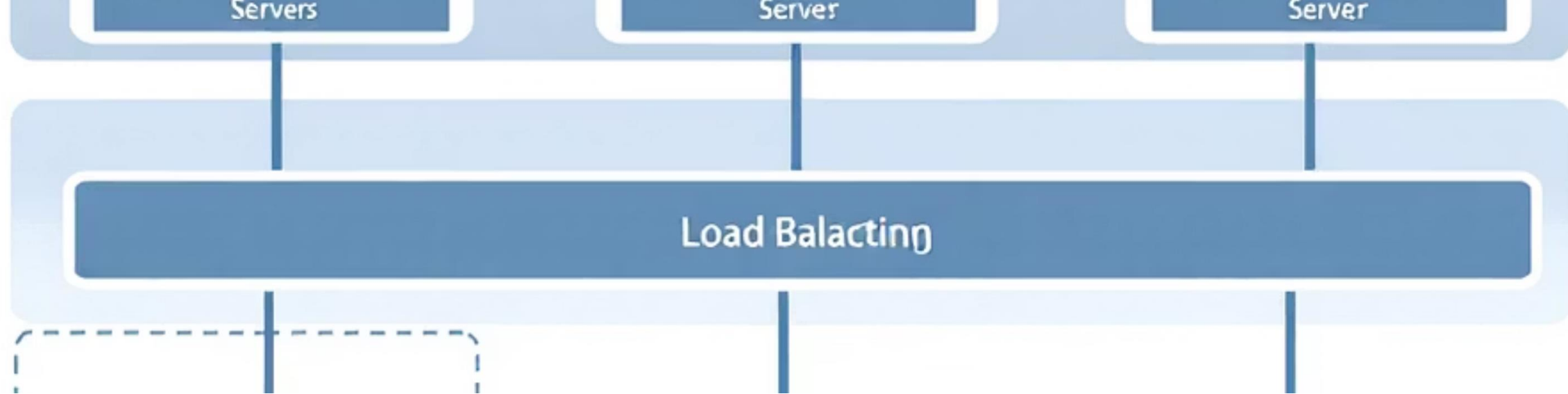
- 사용자를 대신하여 동작하는 모든 도구
- 주로 웹 브라우저가 이 역할 수행
- 개발자 도구 및 디버깅 프로그램도 포함

클라이언트의 역할

- 항상 요청을 보내는 개체 (서버가 될 수 없음)
- 문서 수신 및 파싱(구문분석) 수행
- CSS와 같은 레이아웃 정보와 결합
- 스크립트 실행을 통한 추가 리소스 요청



브라우저는 HTML, CSS, JavaScript를 해석하여 사용자에게 웹 페이지를 표시합니다.



HTTP 기반 시스템: 웹 서버

1

서버의 기본 역할

클라이언트의 요청에 응답하여 문서를 제공하는 시스템

HTTP 응답을 생성하고 클라이언트에게 전송

2

물리적/논리적 구조

논리적으로는 단일 기계처럼 동작하지만, 실제로는 여러 서버의 집합일 수 있음

단일 머신 위에 여러 서버를 호스팅하는 것도 가능

3

기능적 구성

- 로드 밸런싱을 통한 트래픽 분산
- 캐시 서버를 통한 응답 속도 향상
- 데이터베이스 서버와의 연동
- e-커머스 서버 등 전문화된 기능 제공

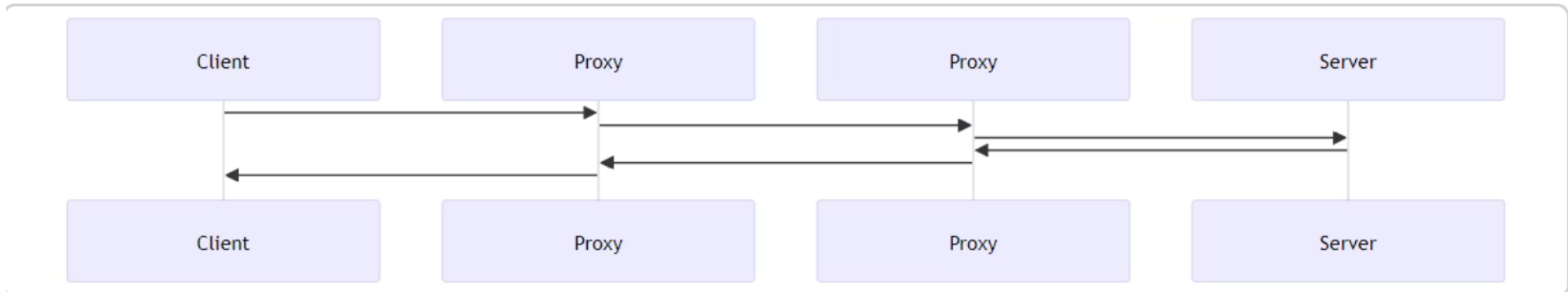
HTTP 기반 시스템: 프록시

프록시의 정의

웹 브라우저와 서버 사이에서 HTTP 메시지를 중계하는 애플리케이션 계층의 컴퓨터/머신

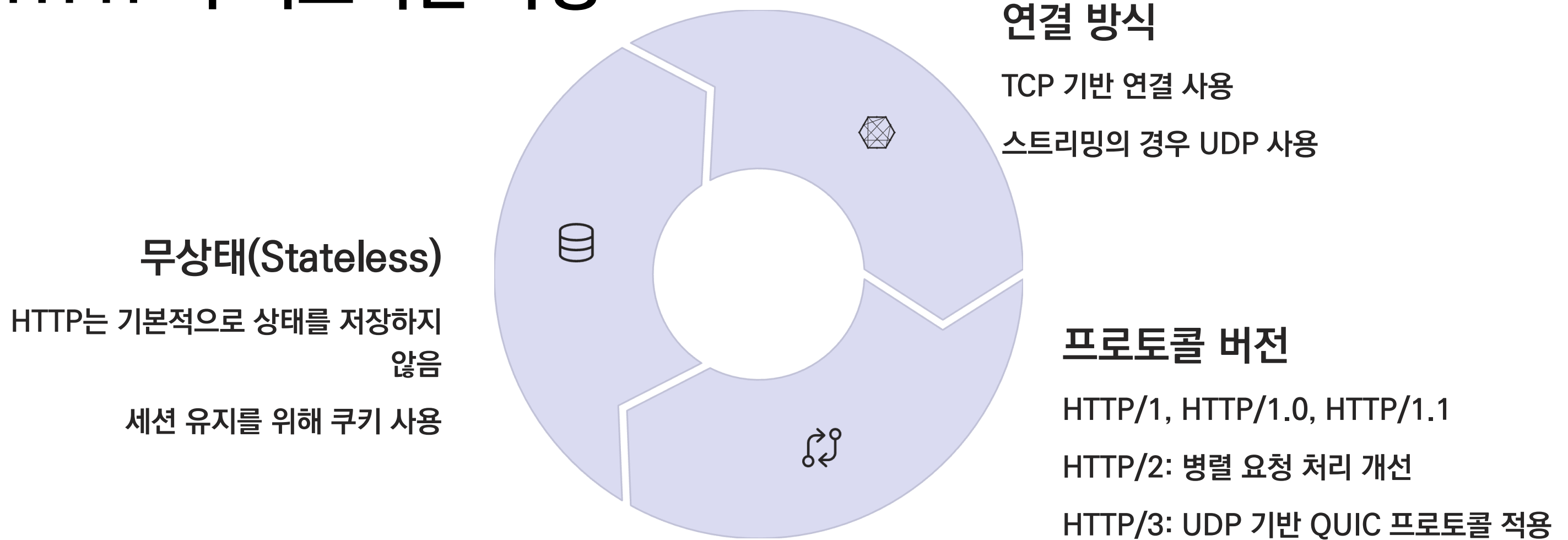
프록시의 주요 기능

- 캐싱: 공개/비공개 캐시 제공 (브라우저 캐시 등)
- 필터링: 바이러스 검사, 유해 콘텐츠 차단
- 로드 밸런싱: 여러 서버에 요청 분산
- 인증: 리소스 접근 제어
- 로깅: 통신 이력 정보 저장



프록시 서버는 클라이언트와 서버 사이에서 중간자 역할을 수행하며 다양한 부가 기능을 제공합니다.

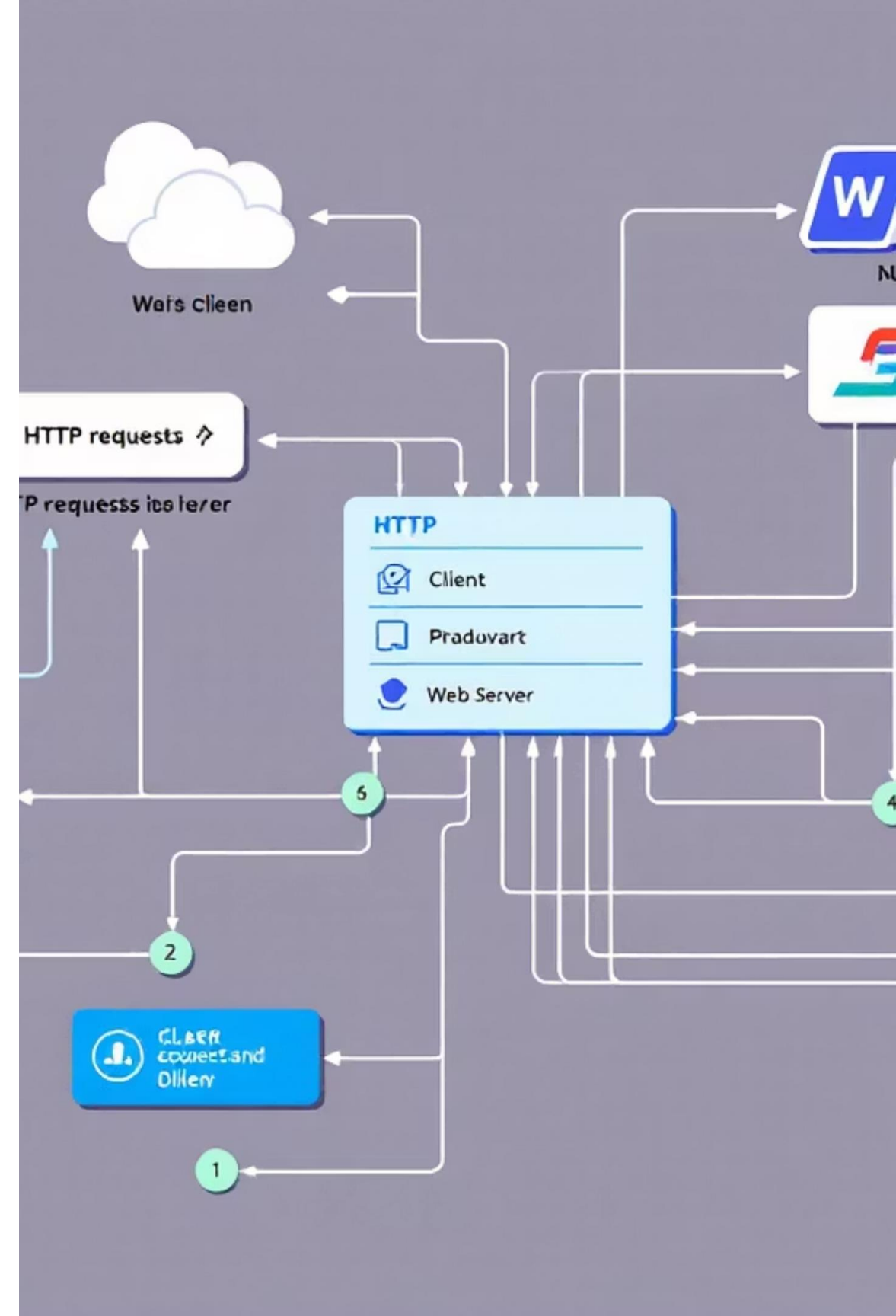
HTTP의 기초적인 특성



HTTP는 계속해서 발전하고 있으며, 웹 개발자는 이러한 변화를 이해하고 적용하여 더 효율적인 웹 애플리케이션을 개발할 수 있습니다.

HTTP 프로토콜 흐름의 이해

개발자와 IT 전문가를 위한 HTTP 프로토콜의 핵심 개념과 메시지 구조를 알아보겠습니다.



HTTP 통신의 4단계 흐름



TCP 연결 열기

클라이언트가 새 연결을 열거나 기존 연결을 재사용하여 서버와 통신을 시작합니다.



HTTP 메시지 전송

요청 메시지를 서버로 전송합니다.



응답 메시지 읽기

서버가 전송한 응답 메시지를 읽고 처리합니다.



연결 닫기 또는 재사용

HTTP/1.1 이상에서는 연결을 재사용할 수 있습니다.

HTTP 세션: 3단계 과정

TCP 연결 수립

클라이언트가 서버와 TCP 연결을 수립합니다.

요청 전송 및 대기

클라이언트는 요청을 전송한 뒤 응답을 대기합니다.

서버 응답 처리

서버는 요청을 처리 후 상태 코드와 데이터를 함께 응답합니다.

HTTP/1.1부터는 연결을 닫지 않고 2~3단계를 반복합니다.

HTTP 메시지의 두 가지 형태

HTTP/1.1 및 초기 버전

사람이 읽을 수 있는 텍스트 형태로 구성되어 있어 디버깅과 분석이 용이합니다.

- 일반 텍스트 형태
- 헤더와 본문이 명확히 구분
- 개발자가 쉽게 해석 가능

HTTP/2

성능 향상을 위해 이진 구조의 프레임으로 임베드되어 있어 직접 읽기 어렵습니다.

- 이진 프레임 구조
- TLS 등과 함께 사용
- 성능 최적화에 집중

HTTP 프로토콜의 역사적 배경

1989년 - 제안

1

Tim Berners-Lee가 CERN에서 인터넷을 통한 하이퍼텍스트 시스템 제안

2

1991년 - 완료

HTML, HTTP, 첫 번째 브라우저 WorldWideWeb, httpd 서버 개발 완료

웹의 기본 구성 요소인 HTML, HTTP, 브라우저, 서버가 모두 이때 탄생했습니다.

HTTP/0.9 – 최초의 HTTP

GET /mypage.html A very simple HTML page

- 단순한 구조
GET 메소드만 지원, 헤더 없음
- 제한된 기능
HTML 문서만 전송 가능, 상태 코드 없음
- 기본 통신
전체 URL 불필요, 연결 후 즉시 종료

HTTP/1.0 - 실용적 진화

```
GET /mypage.html HTTP/1.0
User-Agent: NCSA_Mosaic/2.0 (Windows 3.1)
200 OK
Date: Tue, 15 Nov 1994 08:12:31 GMT
Server: CERN/3.0 libwww/2.17
Content-Type: text/html
```

1

버전 정보

HTTP/1.0 표시

2

상태 코드

200 OK 등 추가

3

헤더 도입

메타데이터 전송

4

다양한 파일

이미지 등 지원

HTTP/1.0 통신의 한계



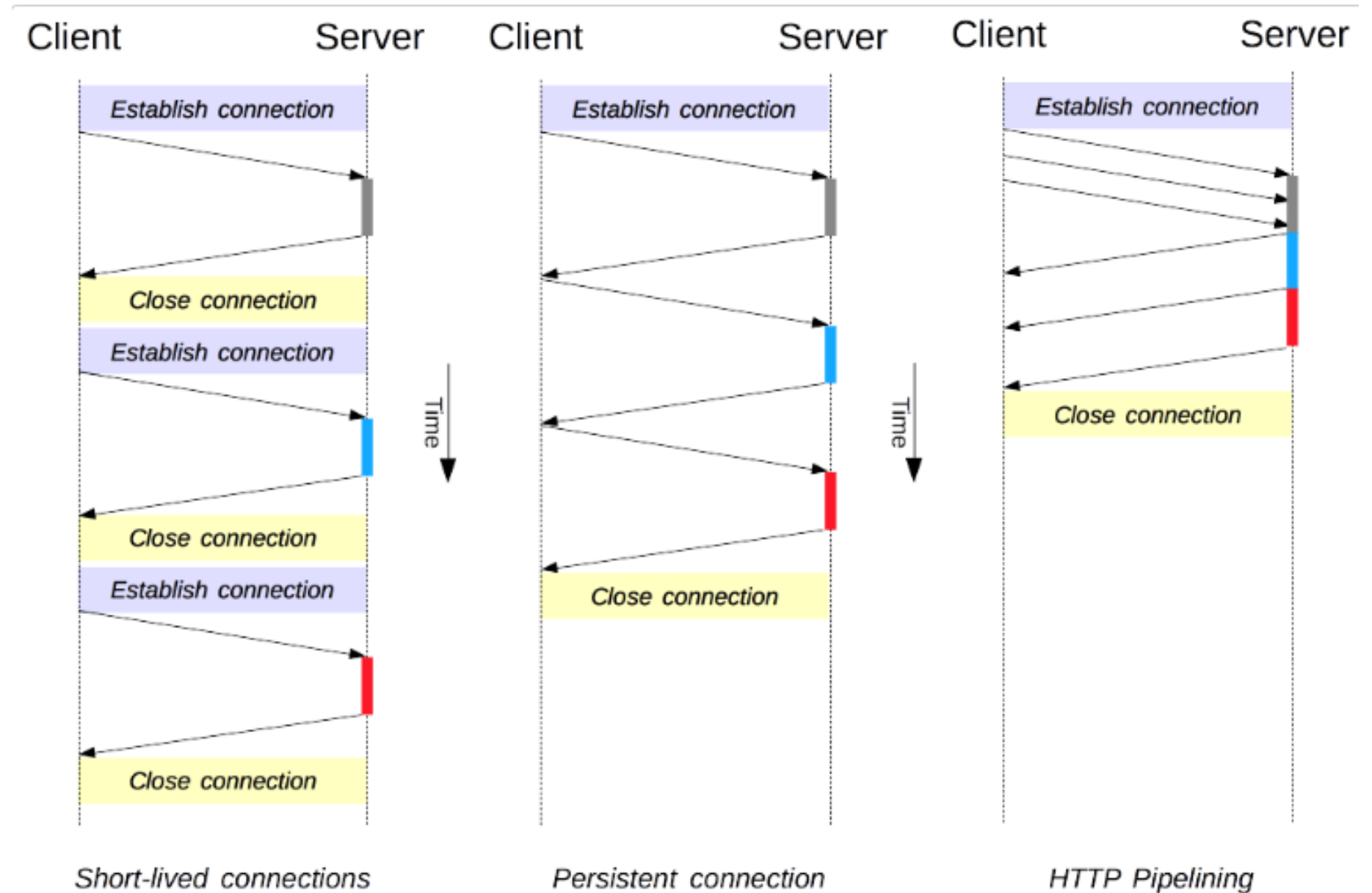
비효율적 연결

한 번의 TCP 연결로 하나의 오브젝트만 전송 후 연결 종료
반복적 연결

- HTML 문서 + 이미지 4개 = TCP 연결 5번
- 매번 3-way 핸드셰이크 필요
- 연결 설정 오버헤드 발생

HTTP/1.x의 커넥션 관리

- 단기 커넥션
 - HTTP/1.0 기본
 - 3-way handshaking
 - 단점
 - 매번 새로운 연결
- 영속적인 커넥션
 - Keep-alive connection(54p)
 - 핸드셰이크 비용 절약
 - 성능향상
- HTTP 파이프라이닝
 - 응답을 기다리지 않고 요청을 연속적으로 보냄



HTTP/1.1 – 연결 재사용의 혁신

1

연결 재사용

Keep-Alive로 하나의 TCP 연결에서 여러 오브젝트 전송

2

파이프라이닝

응답을 기다리지 않고 연속 요청 전송

3

Host 헤더

동일 IP에서 여러 도메인 호스팅 가능

4

인코딩 지원

gzip 등 압축 기능 추가

현재까지 가장 널리 사용되는 HTTP 버전으로, 웹 통신의 표준입니다.

HTTP/2 프레임: 개선점

헤더 압축

HTTP/1.x에서 압축되지 않던 헤더가 압축 가능해졌습니다.

중복 제거

연속된 메시지의 비슷한 헤더 구조 반복 전송 문제를 해결했습니다.

멀티플렉싱

여러 스트림을 하나로 묶어 TCP 연결을 효율적으로 사용합니다.

HTTP/2: 성능 혁신

이진 프로토콜

텍스트 기반에서 이진 기반으로 변경하여 파싱 효율성 향상

다중화 지원

단일 연결에서 여러 요청을 병렬로 처리하여 성능 개선

헤더 압축

HPACK 압축으로 반복적인 헤더 정보의 오버헤드 감소

서버 푸시

클라이언트 요청 전에 필요한 리소스를 미리 전송

2015년 표준화된 HTTP/2는 2022년 기준 약 50%의 사용률을 보이며, 최신 브라우저와 서버에서 자동 지원됩니다.

HTTP 기술의 지속적 발전



보안 강화

1994년 SSL 1.0 도입으로 HTTPS 암호화 통신 시작



REST API

2000년 REST 모델로 웹 애플리케이션 API 표준화



실시간 통신

Server-Sent Events와 WebSocket으로 Push 기능 구현

HTTP는 단순한 문서 전송에서 복잡한 웹 애플리케이션 지원으로 진화했습니다.

프로토콜 업그레이드 메커니즘

- Upgrade헤더 필드를 사용하여 이미 설정된 연결을 다른 프로토콜로 업그레이드하는 메커니즘 제공

```
GET /index.html HTTP/1.1  
Host: www.example.com  
Connection: upgrade  
Upgrade: example/1, foo/2
```

- 웹소켓은 항상 HTTP, HTTPS 연결을 업그레이드하여 구현

```
WebSocket = new WebSocket("ws://destination.server.ext", "optionalProtocol");  
% “wss://” URL 스키마로 웹 소켓 연결 오픈 가능
```

- 웹 소켓을 처음부터 만들 때
Connection: Upgrade
Upgrade: websocket

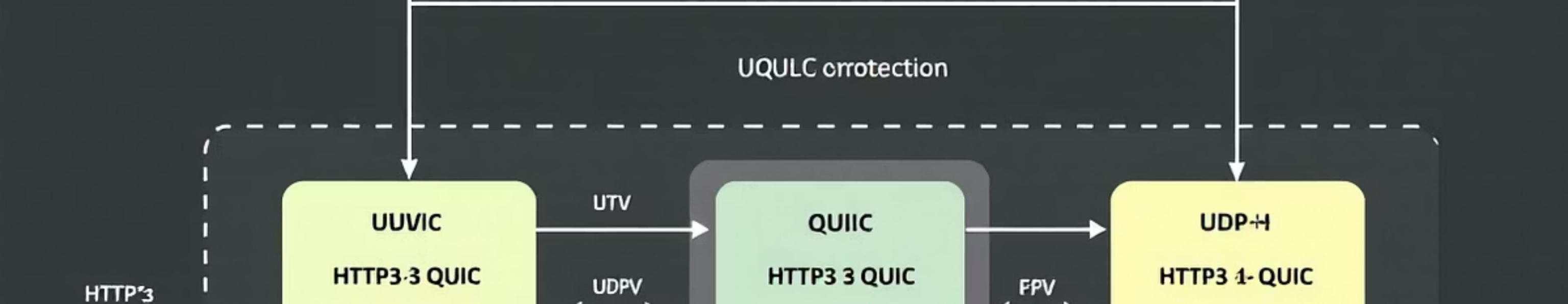
CDN과 리소스 분리



CDN 메커니즘

HTTP/2는 CDN(Content Delivery Network)을 통해 신분 증명과 리소스 위치를 분리합니다.

전 세계에 분산된 서버 그룹이 사용자와 가장 가까운 위치에서 콘텐츠를 제공하여 응답 시간을 단축시킵니다.



HTTP/3: 차세대 프로토콜

QUIC 기반

UDP 기반의 QUIC 프로토콜로
Zero RTT 실현

연결 유지

WiFi와 셀룰러 간 전환 시에도 연결
상태 유지

빠른 복구

패킷 손실 시 신속한 복구로 안정성
보장

HTTP/3 지원 현황

브라우저 지원

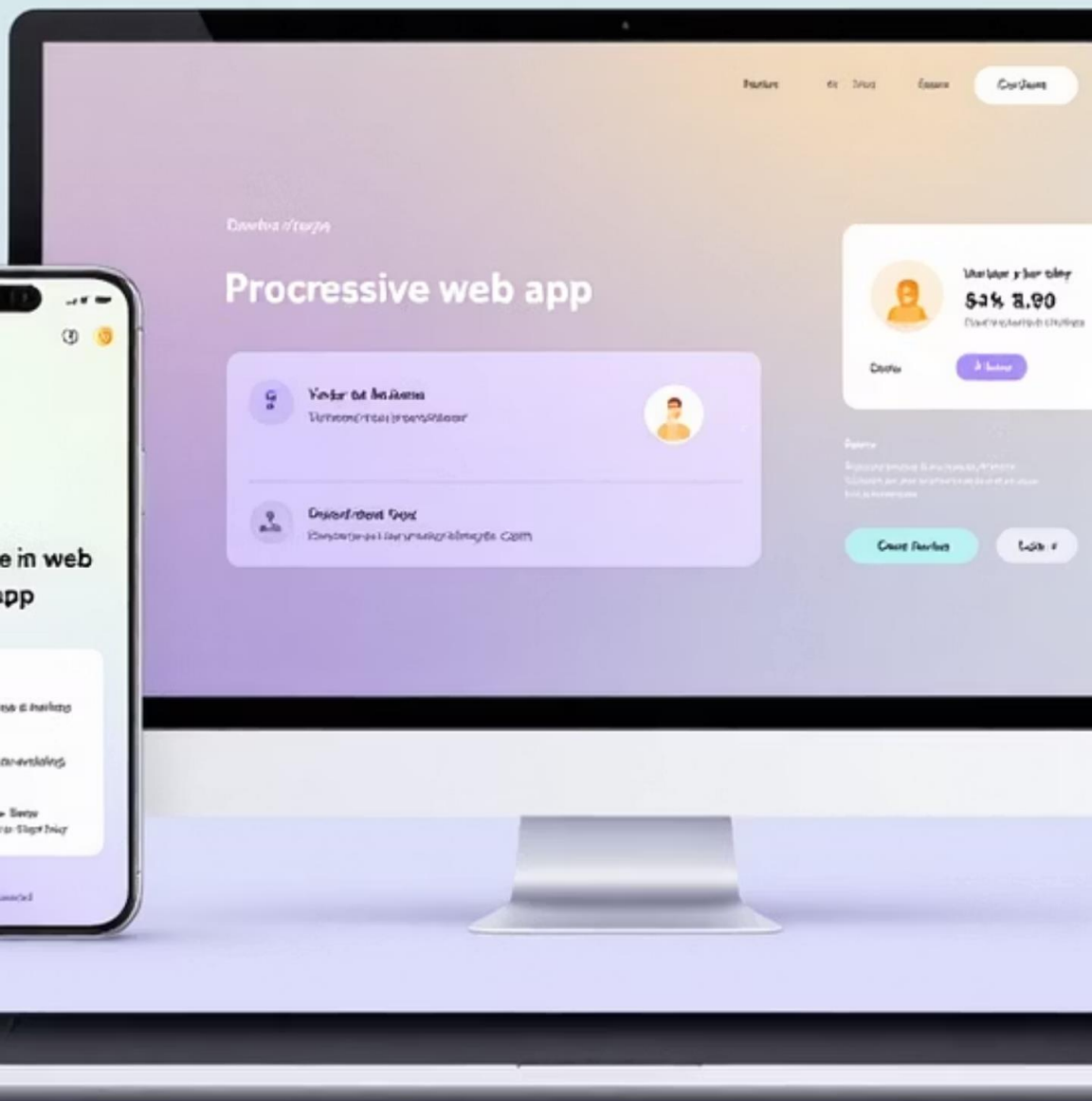
- Chrome: 2019년 12월부터
- Safari, Edge, Firefox: 2020년부터

웹서버 지원

- NGINX, Microsoft IIS

주요 적용 사이트

- Google, YouTube, Netflix
- 네이버, 나무위키
- Facebook, Instagram
- Zoom Video Communications



프로그레시브 웹 앱 개념

PWA는 웹 앱과 네이티브 앱의 장점을 결합한 진보된 웹 애플리케이션 개발 방식입니다.

50%

모바일 사용자

앱을 다운로드하지 않지만 100개 이 모바일 환경 사용자가 네이티브 앱 선호
상의 웹사이트 방문

87%

네이티브 앱

PWA 핵심 특징



오프라인 지원

네트워크 연결 없이도 기본 기능 제공



푸시 알림

사용자 재참여를 위한 알림 기능



설치 가능

홈 화면에 추가하여 네이티브 앱처럼 사용



반응형 디자인

모든 디바이스에서 최적화된 경험 제공

리소스와 URIs

웹 개발의 핵심 개념인 URI와 HTTP 메시지 구조를 살펴보겠습니다.



리소스와 URI 개념

리소스

URI로 식별하는 HTTP 요청대상입니다.
URI로 표시할 있는 모든 것

URL 예시

- <https://developer.mozilla.org>
- <https://example.com/search?q=URL>

URI 식별자

HTTP 요청 대상인 리소스를 고유하게 식별하는
통일된 문자열입니다.

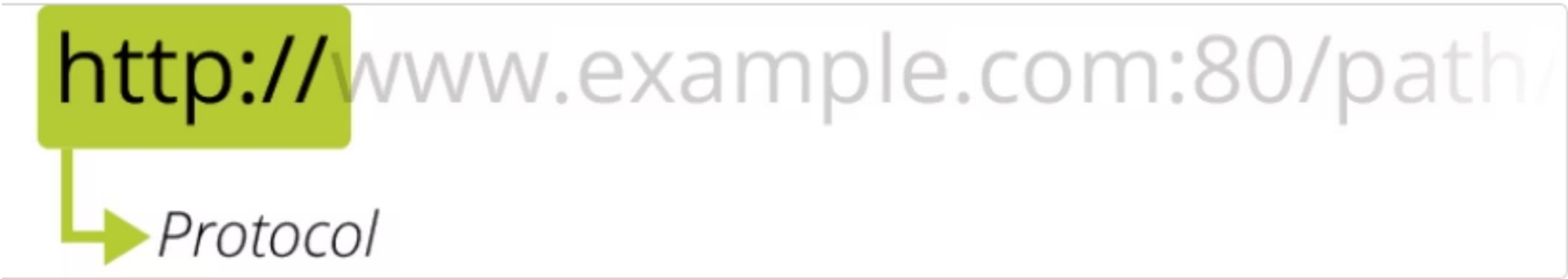
URN 네임스페이스

특정 네임스페이스에서 이름으로 리소스를 식별합니다.

- [urn:isbn:9780141036144](#)
- [urn:ietf:rfc:7230](#)

URI 문법: URI 스키마

프로토콜 명시



스키마 유형	설명
http/https	하이퍼텍스트 전송 프로토콜
ftp	파일 전송 프로토콜
ws/wss	웹소켓 연결
mailto	전자메일 주소
tel	전화번호

⚠️ 보안상의 이유로 FTP를 통한 하위 리소스 로드 지원이 제거되고 있습니다.

le.co

URI 문법: 도메인과 포트 구조

도메인 이름 (또는 IP 주소)

네임스페이스를 관리하는 도메인 또는 IP 주소로 서버를 식별합니다.

tp://www.example.com:80/path/to/my

→ Domain Name

표준 포트

HTTP는 80번, HTTPS는 443번 포트를 기본으로 사용합니다.

com:80/path/to/myfile.html?key1=valu

→ Port

표준 포트가 아닌 경우 반드시 포트 번호를 명시해야 합니다.

:80/path/to/myfile.html?key1=value



Path to the file

URI 문법: 경로

URI 경로는 웹 서버의 리소스 위치를 나타냅니다.

과거에는 물리적 파일 경로였지만, 현재는 웹 서버에서 추상화된 처리 방식을 사용합니다.

nl?key1=value1&key2=value2#So



Parameters

URI 문법: 쿼리

쿼리 문법

키/값 쌍을 &로 구분하여 추가 매개변수를 전달

서버 처리

각 웹 서버마다 고유한 매개변수 처리 규칙 보유

e2 #SomewhereInTheDocument



URI 문법: 프래그먼트(Fragment)

프래그먼트는 리소스 내부의 특정 부분을 가리키는 앵커 역할을 합니다.
HTML에서는 스크롤 위치를, 미디어에서는 재생 시점을 나타냅니다.

❗ 프래그먼트(# 뒤의 부분)는 서버로 전송되지 않습니다.

데이터 URI

1 기본 문법

`data:[mediatype][:base64],data`
% Mediatype이란 MIME type

2 텍스트 예시

`data:,Hello%2C%20World!`

Hello, World!

%2C 의미: ,
%20 의미: (공백)

3 Base64 인코딩 버전

`data:text/plain;base64,SGVsbG8slFdvcmxklQ==`

4 Base64 인코딩 문제

길이제한(65529문자로 제한)
피싱 보안 문제가 있어 탐색은 차단